EnergyGate: Defending Battery-Powered IoT Nodes Against Post-Quantum Handshake Exhaustion Attacks

Anonymous Authors
Under double-blind review

***Abstract*—A battery-powered field device that accepts post-quantum handshakes pays for every request it is asked to check, whether or not the request is genuine. We measure that cost on commodity ESP32 microcontrollers running ML-KEM (FIPS 203) and ML-DSA (FIPS 204) over a broadcast ESP-NOW mesh, through a calibrated high-side INA219 rig. At NIST level 3 the responder's share of one handshake costs ≈14.9 mJ above idle; a forged HELLO carrying a structurally valid signature forces a full ML-DSA-65 verification (≈3–3.5 mJ) before it fails. Undefended, a forger resending one signature at $5\,\mathrm{s}^{-1}$ adds $+14.7 \pm 0.9$ mW (120 s, $n = 5$, 95 % CI, repeated in a second session) and an enrolled insider $+91.2 \pm 3.3$ mW. We present *EnergyGate*, which decides at the first fragment of a HELLO—before reassembly and before any public-key work—whether to spend, challenge or drop, using an eight-feature link score and a joule token bucket. Against attackers that neither answer its broadcast cookie nor fool its score it removes the drain (within ≈3 mW of idle) without locking out the real gateway; an adaptive attacker that keeps its score high still drains $+25.5 \pm 3.0$ mW. Against attackers that echo the cookie under the challenged (epoch, seq) it does much less: a forger still drains $+10$ to $+20$ mW under a cookie (range over three sessions), and an insider who echoes drains $+69.0 \pm 9.4$ mW under the gate against $+73.2 \pm 4.1$ mW undefended (600 s, $n = 3$). The joule budget bounds what the node *charges*, not what it spends: it roughly halved a forger's echo drain once it emptied, but only because our placeholder charge overcosts a forged verify ≈6–7×, and it never bound against the insider, whose admissions stayed below its cap. The defences protect energy, not availability: the real gateway's success tracks the attacker's radio load. Retrained on recorded traces and evaluated by session, the gate's tree reaches AUC 0.95 and catches most adaptive traffic the original model misses; it is now deployed in the firmware, though its power effect on the rig is not yet measured.**

***Index Terms*—post-quantum cryptography, ML-KEM, ML-DSA, energy exhaustion, denial of service, ESP32, ESP-NOW, admission control**

I. Introduction

Post-quantum key exchange is arriving on constrained devices. ML-KEM and ML-DSA [1], [2] run on a commodity ESP32 in milliseconds to tens of milliseconds per operation (Table II), so the question is no longer whether a microcontroller *can* run a signed post-quantum handshake, but what it costs to let anyone *ask* it to.

Field devices in distribution grids, pipelines and environmental monitoring often run on a battery and share a broadcast radio. A post-quantum handshake is an expensive operation, unauthenticated until verified, triggered by a message anyone in range can send. At NIST level 3 a single HELLO is 4517 bytes—20 ESP-NOW frames—and checking it means an ML-DSA-65 verification; admitting it commits the device to an encapsulation, a signature and a 19-frame response. The cryptography protects the message. Nothing in it protects the battery.

Our motivating scenario follows the 2015 attack on the Ukrainian grid [3], whose chain ran from phishing through malicious attachments to the field network and physical access. We study the field-network step: an attacker within radio range of a battery-powered node who wants to shorten its life or deny service without breaking any cryptography. Our contributions:

- **C1, measured cost.** Energy per post-quantum operation and per handshake on real ESP32 hardware, through a calibrated INA219 rig, at two parameter sets (Sections VI–VII).
- **C2, the attack.** Victim power and legitimate-handshake success for six attacker behaviours under four defences, with five repeats and 95 % CIs (Section VIII).
- **C3, a budget-bounded learned gate.** EnergyGate decides at the first fragment whether to spend, challenge or drop. We show on the rig that its joule bucket caps what the node charges at the predicted rate even when the cookie and the score are both defeated, and that this cuts real drain only where the charge overstates the true cost. It does not stop an authenticated insider who echoes the cookie (Sections V, VIII, IX).
- **C4, the artifact.** Firmware, rig description, calibration data, raw logs and scripts that rebuild every table.

II. Background and Related Work

ML-KEM and ML-DSA. ML-KEM is a module-lattice KEM and ML-DSA a module-lattice signature with rejection-sampled signing, so its signing time is variable [1], [2]. We use PQClean's portable C implementations [4].

ESP-NOW is Espressif's connectionless vendor-frame protocol, at most 250 bytes per frame in v1 [5]. We broadcast, so there is no link-layer retransmission and every station in range hears every frame.

PQC cost on small devices. Gillot *et al.* measure ML-KEM/ML-DSA/SLH-DSA energy on Raspberry Pi and laptop platforms [6] and TLS 1.3 PQC handshakes on Pico W, ESP32 and Pi Zero [7]; Karcz *et al.* time the NIST schemes on an ESP32-C6 [8]; Segatz and Al Hafiz speed up Kyber on the ESP32 [9], and Ben Ahmed *et al.* benchmark post-quantum

signatures on ESP32 and Raspberry Pi [10]. All characterise the cost the device *wants* to pay. None measures the cost an adversary chooses when to impose.

Handshake exhaustion. Lee *et al.* show PQC-TLS prolongs high server CPU load under handshake exhaustion by up to 88× and degrades learned IDSs [11], and Sammo *et al.* make a TLS server pay for an expensive post-quantum signature by algorithm substitution [12]—server CPU in both, not battery. Darzi and Yavuz combine post-quantum security with counter-DoS for spectrum access [13]. Hummen *et al.* protect constrained peers' public-key work with puzzles [14], Tiloca *et al.* harden the DTLS handshake against DoS [15], and client puzzles and cookies for key exchange are well studied [16]–[18]. On a broadcast medium a stateless cookie only stops blind spoofers.

Battery exhaustion. Stajano and Anderson named "sleep deprivation torture" [19]. Martin *et al.* describe *service request* attacks, in which the victim spends energy just deciding whether to honour a request [20]—the attack we measure, here with a post-quantum verify as the decision. Denial of sleep has been studied at the MAC layer [21], at the clustering layer [22] and in battery models [23]. Closest to us, Krentz *et al.* evaluate a symmetric 802.15.4 session-key scheme against injected key-establishment requests [24], and Hristozov *et al.* throttle costly RESTful IoT requests, including from compromised requesters, through a trusted backend [25]. Neither involves post-quantum signatures or a joule budget on the node.

Search. We searched OpenAlex, arXiv and Crossref on 6 Nov 2026 with 24 fixed queries over eight topics (PQC cost on microcontrollers, PQC handshake DoS, battery exhaustion, puzzles and cookies, learned admission, ESP-NOW, energy budgets, join procedures) and screened the 1 542 unique hits by title (queries, raw results and screening in the artifact; DBLP refused automated queries). We found no prior measurement of the battery cost of abusive post-quantum handshake requests on a microcontroller and no defence that bounds it with a joule budget independent of a classifier. Screening was by one author and by title, so this is not a full systematic review.

III. Threat Model

Setting. One battery-powered responder (*field-1*) and one mains-powered gateway (the initiator) share a broadcast ESP-NOW channel. Only the gateway initiates; a real field node never sends a HELLO.

Attacker. Within radio range; transmits arbitrary frames at any rate; holds no legitimate long-term key; spoofs any sender id (ESP-NOW frames are unauthenticated); hears every frame, so it can replay HELLOs and echo any broadcast cookie; and knows the defence, its features, thresholds and budget (Kerckhoffs). Behaviours:

- **FLOOD**: forged HELLOs under the gateway's id with random signature bytes. ML-DSA rejects them during signature unpacking (≈1.7 ms), so they are cheap to reject.
- **FORGE**: one *structurally valid* ML-DSA-65 signature from the attacker's own key, resent under the gateway's id with fresh (epoch, seq). It passes unpacking, so field-1 runs the full verification and fails only at the end. One attacker signature forces a full check on every resend.
- **INSIDER**: HELLOs validly signed with an *enrolled* key (a compromised peer); each admitted HELLO costs a full responder handshake.
- **SLOW_DRIP**: INSIDER at ≈1 HELLO/min, below any fixed rate limit.
- **ADAPTIVE / ADAPTIVE_SPOOF**: knows Energy-Gate's features; keeps `hs_fail` and `dup_pct` at zero and paces `hs_per_s` under the tree's split with AIMD back-off on a heard CHALLENGE.
- **+ECHO**: any of the above, also echoing the cookie from each CHALLENGE it hears.

Out of scope: jamming, physical capture, side channels, breaking ML-KEM/ML-DSA. The attacker wins by maximising victim energy or denying the gateway a session; we report both.

TABLE I
Message sizes and ESP-NOW frames (235 B payload per frame)

Level	HELLO	RESPONSE	CONFIRM
1	3244 B, 14 fr.	3220 B, 14 fr.	48 B, 1 fr.
3	4517 B, 20 fr.	4429 B, 19 fr.	48 B, 1 fr.

IV. System: the Post-Quantum Handshake

A signed-KEM exchange in three messages, on PQClean (commit `0586a824`):

$$I \to R : \mathsf{HELLO} = \text{cookie} \,\|\, n_i \,\|\, pk_{\mathrm{KEM}} \,\|\, \sigma_I(\mathrm{th}_1)$$
$$R \to I : \mathsf{RESPONSE} = n_i \,\|\, n_r \,\|\, ct \,\|\, \sigma_R(\mathrm{th}_2, R, I)$$
$$I \to R : \mathsf{CONFIRM} = n_r \,\|\, \mathrm{tag}$$

with $\mathrm{th}_1 = \mathrm{XOF}(I, R, n_i, pk)$, $\mathrm{th}_2 = \mathrm{XOF}(\mathrm{th}_1, n_r, ct)$, session keys $\mathrm{XOF}(ss, \mathrm{th}_2)$ and SHAKE256 as XOF. Static provisioned ML-DSA keys authenticate; an ephemeral ML-KEM key agrees. Session keys drive AES-256-GCM. The cookie lies outside the signed transcript so that, after a CHALLENGE, the initiator resends the same signed HELLO without signing again. The protocol is host-tested against forged signatures, wrong keys, tampered fields and replays, but **not formally verified**.

Level 1 is ML-KEM-512 + ML-DSA-44; level 3 (default) is ML-KEM-768 + ML-DSA-65. Level 5 does not fit: ML-DSA-87 signing needs more stack than the largest free heap block (110 580 B). Table I gives message sizes.

Resource bounds. The reassembler tracks at most 8 streams with a 3 s timeout. All post-quantum work runs in one task with a reserved stack (90 112 B at level 3); while busy, admissions are refused (`busy_drop`), which caps the crypto rate. **Replay guard:** per sender, the last 8 verified epochs with their highest verified sequence number; unverified HELLOs never advance it.

V. EnergyGate

Placement. On field-1, whose battery is at risk. Inbound HELLO checks run: header sanity, replay/freshness, allow-list, EnergyGate with the joule budget, then reassembly and ML-DSA verification. EnergyGate decides on the *first fragment*. DROP is silence; the radio still receives the remaining fragments, and that residual cost is measured, not assumed away.

Score. A depth-4 decision tree, exported to C, maps eight link features over a rolling 5 s window to the probability that the sender is genuine: `hs_per_s`, `hs_fail`, `rssi_mean`, `rssi_var`, `loss_pct`, `dup_pct`, `frag_complete_pct`, `battery_pct`. The model used in every power run was trained on SIMULATED traces; a tree retrained on recorded traces has since replaced it (Section IX).

Joule budget. A token bucket of capacity B refilling at r J/s; an admitted handshake is charged c_h, a challenge c_c. Every run used the PLACEHOLDER values $B = 20$ J, $r = 0.05$ J/s, $c_h = 0.02$ J, $c_c = 0.5$ mJ. Measured values (Section VII): field-1's share of a handshake is ≈14.9 mJ (so c_h overcharges it by 1.3×), a failed FORGE verify ≈3 mJ measured directly ($14.7\,\text{mW}/4.97\,\text{s}^{-1}$, Section VIII; ≈6–7× overcharge), an echoing insider's admit ≈30 mJ (power over admit rate, so it includes radio work; c_h *under*charges it), and $c_c \lesssim 0.4$ mJ (an upper bound from the noise floor). Since the bucket never spends more than it holds,

$$\sum_{\text{admits in } [0,T]} c \;\le\; B + rT, \qquad \lim_{T\to\infty} \bar{P}_{\text{charged}} \le r, \quad (1)$$

for *any* admission sequence, whatever the score says. It caps what is *charged*; it caps real drain only when c_h is at least the true cost, and it does not cover reception of frames before the gate decides.

Policy v2 (default), with score p, thresholds 0.7/0.4: (1) $p \ge 0.7$ and budget covers $c_h \to$ SPEND; (2) a valid cookie returned and budget covers $c_h \to$ SPEND whatever the score; (3) budget cannot cover one handshake → DROP; (4) $p \ge 0.4 \to$ CHALLENGE; (5) otherwise CHALLENGE charged c_c, DROP only if c_c cannot be paid. **Policy v1** (ablation) drops at rule 5 and needs $p \ge 0.4$ at rule 2.

Baselines. NONE; RATELIMIT (one admission per sender per 10 s); COOKIE (challenge every sender until it echoes a cookie; no model, no budget).

VI. Measurement Setup

Four ESP32-WROOM-32 devkits (ESP32-D0WD-V3, 240 MHz): field-1 (victim), gateway, attacker, monitor. PlatformIO 6.2.0, espressif32 7.1.3. Energy builds strip the demo peripherals. There is no battery: `battery_pct` is fixed at 100 (SIMULATED).

Rig. Field-1 is powered from a 5 V adaptor *only* through a high-side INA219 (0.1 Ω shunt, 0.1 mA/bit), read by the monitor at ≈989 samples/s over 400 kHz I^2C. Field-1 raises GPIO4 around each measured operation; the monitor reports per-interval duration, energy and peak current, and a true 1 s mean power.

TABLE II
PRIMITIVE TIMINGS, RADIO OFF (MS; MEDIAN OF FOUR BOARDS, 5 SESSIONS EACH; SIGNING POOLED, $n = 116$)

Algorithm	keygen	encaps/sign	decaps/verify
ML-KEM-512	5.93	6.05	7.35
ML-KEM-768	8.59	9.88	11.60
ML-KEM-1024	13.14	14.37	16.46
ML-DSA-44	15.36	48.8 (p10 24.1, p90 92.4)	15.97
ML-DSA-65	26.00	63.4 (p10 37.1, p90 152.7)	26.25
X25519†	397.45	403.76	—
ECDSA-P256†	191.16	89.04	202.56

†mbed TLS generic C; not a fair PQC-vs-classical comparison.

Calibration. Seven loads from 0 to 232 mA against a hobby-grade DT830D multimeter (not a calibrated reference) fit $I_{\text{INA}} = 1.0367\, I_{\text{ref}} - 0.428$ mA; after correction residuals are within 0.6 % over 57–232 mA and ≈0.17 mA below 20 mA. Systematics are quoted separately: the 3.67 % gain difference between the two instruments is unresolved and affects absolute joules (C1) but cancels in ratios, and the INA219's bus voltage reads 1.4 % (5 mA) to 3.1 % (232 mA) above the DT830D. Raw logs stay uncorrected; correction is applied in analysis.

Supply. The adaptor's voltage at field-1 is not fixed: it ranged 4.75–4.93 V across sessions and sometimes stepped within a run. Field-1 draws a near-constant current (117–120 mA idle), so its power follows the supply, and a step between baseline and attack reads as extra power. We check each run's bus shift. Long-window means in this paper had shifts of at most ≈0.01 V; where a figure depends on it we say so.

Method note. The monitor's instantaneous current sample aliases against a 5 Hz attacker (one FORGE run read ≈0 mW, another +125 mW). All power results use the 1 s mean.

VII. Results: Cost of the Handshake (C1)

Deterministic timings agree across all four boards within 0.6 % (Table II); signing is rejection-sampled, so we report its pooled distribution (ML-DSA-65 max 394.8 ms). Table III gives energy per operation (radio off, $n = 20$ per op; idle 225 mW).

At level 3, field-1's share of a handshake is verify + encaps + sign $= 3.23 + 1.10 + 10.6 \approx \mathbf{14.9}$ mJ above idle; both sides together cost ≈31 mJ. On the live mesh with the radio always on (field-1 idles at ≈598 mW), one legitimate handshake costs 9–65 mJ above idle (80–560 ms, dominated by signing variance); one admitted FORGE HELLO ≈3.45 mJ per marked operation (and ≈3 mJ as mean extra power over verify rate in a later session: two methods, two sessions); one admitted FLOOD HELLO is not measurably different from zero. Peak crypto current is ≈71 mA. The idle radio dominates absolute drain, so any battery-life claim depends on a duty-cycle assumption we state rather than hide: our victim never sleeps.

TABLE III
ENERGY PER OPERATION, RADIO OFF (MEAN, $n = 20$). "ABOVE IDLE" SUBTRACTS 225 MW × DURATION

Operation	ms	total mJ	above idle mJ
ML-KEM-768 keygen	8.6	2.91	0.98
ML-KEM-768 encaps	9.7	3.27	1.10
ML-KEM-768 decaps	11.5	3.93	1.34
ML-DSA-44 sign	44	15.4	5.4 ± 1.3
ML-DSA-44 verify	16.2	5.57	1.93
ML-DSA-65 keygen	26.0	9.05	3.19
ML-DSA-65 sign	86	30.0	10.6 ± 4.1
ML-DSA-65 verify	26.2	9.14	3.23
X25519 shared secret†	476	150	43.3
ECDSA-P256 sign†	115	36	9.9

TABLE IV
E5 ENERGY GRID (FIRMWARE `fe253e6`, 25–26 OCT, BEFORE THE BOUND COOKIE): EXTRA VICTIM POWER AND LEGITIMATE SUCCESS

Defence	Attack	Extra power (mW)	Legit success
none	FLOOD	+1.8 ± 2.2	0.71 ± 0.23
	INSIDER	**+91.2 ± 3.3** (+15.2 %)	0.21 ± 0.15
	SLOW_DRIP	+0.2 ± 0.6	0.96 ± 0.11
	FORGE	**+19.0 ± 1.4** (+3.2 %)	0.28 ± 0.24
	ADAPTIVE	**+28.9 ± 2.3**	0.30 ± 0.30
	ADAPT._SPOOF	−0.4 ± 1.8	0.65 ± 0.64
ratelimit	FLOOD/INS./DRIP	−2.2/ − 1.4/ + 0.4	**0.00**/0.87/0.81
cookie	FLOOD/INS./DRIP	+1.8/ + 1.4/ − 0.1	0.79/0.77/1.00
	FORGE/AD./A.S.	+0.8/ + 1.2/ − 1.4	0.24/0.53/0.52
gate v2	FLOOD/INS./DRIP	+1.7/ + 1.1/ + 0.7	0.75/0.82/1.00
	FORGE/A.S.	+1.4/ − 0.4	0.23/0.47
	ADAPTIVE	**+25.5 ± 3.0**	0.45 ± 0.33
gate v1	FLOOD/INSIDER	−2.6/ − 3.0	**0.04/0.02**

VIII. RESULTS: ATTACKS AND DEFENCES (C2, C3)

Each cell of Table IV attacks for 120 s at 5 HELLO/s (SLOW_DRIP: ≈1/min). Extra power is field-1's mean power over the attack minus the minute before; $n = 5$, 95 % CI. Legit success is the gateway's completed/started handshakes during the attack. This grid ran on 25–26 Oct on firmware `fe253e6` (gate v1 row `daedfd1`; rate sweep and 30-min SLOW_DRIP `3879f7a`), before the cookie was bound; field-1's only functional change since is that binding, and the no-echo FORGE cell was rerun on the later firmware at +14.7 ± 0.9 mW, then +14.1 ± 2.2 mW in a third session. We quote +14.7; the grid's +19.0 came from an earlier session with different board placement. Absolute values (power, idle, legit success) did not carry across sessions—legit success under FORGE was 0.23–0.28 here and 42–58 % in the rerun—so we compare only within a session.

Findings. (i) FORGE, not FLOOD, is the cheap-attacker/costly-victim attack: one attacker signature forces a ≈3–3.5 mJ verify per resend, while random signatures cost nothing measurable. (ii) INSIDER is the largest undefended drain and starves the gateway; against an insider that does *not* echo, every defence removes the drain (an echoing insider is not stopped; see below), but RATELIMIT does so by also locking out the gateway under FLOOD (0 %). (iii) Gate v1 locks out the gateway (2–4 %): forged and real HELLOs share a sender id and look alike to window features, so dropping low scores drops both. Gate v2's cookie rule fixes that. (iv) ADAPTIVE evades the score and still drains +25.5 mW under gate v2. (v) In a rate sweep (5/20/50 s^{-1}, $n = 3$) INSIDER's undefended drain is flat (+94 to +96 mW) because the single crypto slot saturates at 5/s; the gate holds it near +1.5 mW at every rate. A fast FLOOD costs no energy but drops legit success from ≈0.5 to ≈0.07 through airtime and reassembly contention—a different denial than the energy one. (vi) A 30-min SLOW_DRIP shows no measurable extra power.

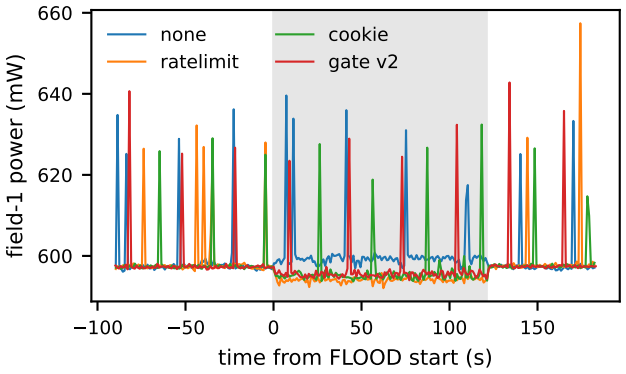


Fig. 1. Field-1's corrected power under FLOOD, one rig run per defence (paper/data/E5_rig, rep 1, 25 Oct). The four traces sit on top of one another within noise, which is itself the finding: FLOOD costs no measurable energy under any defence (Table IV).

FORGE + ECHO (original run, unbound cookie) drew +15.1 ± 2.4 mW under COOKIE and +16.0 ± 0.5 mW under gate v2 ($n = 5$), ≈540 forged HELLOs spent per run, legit success ≈0.13. The cookie as built then was *reusable*: bound to neither the HELLO's nonce nor its (epoch, seq), so ≈30 echoes admitted every later resend of the cached template, and gate v2's rule 2 let them through on top of that.

The cookie now binds to (sender, epoch, seq, window); the fix makes a real echo dearer, not free. Binding it closes the *reuse*, but a first attempt broke the real gateway too—its own cookie retry also used a fresh seq, so a bound cookie never verified for it either. Fixed by having the gateway keep the challenged (epoch, seq) on retry, and by giving the attacker an `ECHO same` mode that does the same (real echo, not the old bug). Rerun on the INA219 (53 runs, firmware `7afdea4`, Table V): with no attack the real gateway now gets in 128/132 times. COOKIE stops a non-echoing or fresh-seq attacker (within ≈1 mW of idle, against +14.7 ± 0.9 mW undefended). A same-seq echo is *not* stopped, at the cost of one echo per HELLO (≈4.6 s^{-1}, roughly double the frames) instead of ≈30 cheap echoes per run. **Dearer for the attacker, not closed.**

A same-seq echo across sessions. Table VI gives every 900 s same-seq FORGE echo run. Under COOKIE the drain

TABLE V
BOUND COOKIE, FIRMWARE 7AFDEA4, 30–31 OCT (ONE SESSION)

Defence	Attacker	Window	Extra power (mW)	Admits/s
gate v2	none (control)	900 s	$+0.2 \pm 0.4$	0.03
none	FORGE, no echo	120 s	$+14.7 \pm 0.9$	4.97
cookie	FORGE, no echo	120 s	-0.7 ± 0.3	0.05
gate v2	FORGE, no echo	120 s	-0.6 ± 0.8	0.07
cookie	fresh-seq echo	120 s	-1.2 ± 2.3	0.05

TABLE VI
SAME-SEQ FORGE ECHO, 900 S, $n = 3$ PER CELL, MEAN $\pm$ 95 % CI. GATE V2 IS SPLIT AT THE BUCKET'S FIRST DROP (BEFORE $\rightarrow$ AFTER); IT EMPTIED AT +412–633 S (30–31 OCT) AND +438–466 S (1–2 NOV)

Session	Defence	Extra power (mW)	Admits/s
30–31 Oct	cookie	$+13.0 \pm 1.0$	4.71
	gate v2	$+12.2 \pm 4.3 \rightarrow +5.5 \pm 2.4$	$4.50 \rightarrow 2.42$
1–2 Nov	cookie	$+19.7 \pm 0.9$	4.54
	gate v2	$\approx +18.3 \rightarrow \approx +9.4$	$4.56 \rightarrow 2.42$
2 Nov	cookie	$+9.7 \pm 3.1$	4.57

was +13.0, +19.7 and +9.7 mW in three sessions, with the same field-1 firmware, the same placement and nearly the same admit rate. The forger with no defence repeated (+14.7 and +14.1 mW), so the spread is specific to the echo. We have not found its cause (Section X). We therefore quote the cookie figure as a range, +10 to +20 mW, and compare cookie with gate only within a session.

The joule budget caps what field-1 charges, and here that cut the real drain. In both sessions with gate v2 runs, the bucket emptied and admissions settled at 2.42 s^{-1}, near the refill bound $r/c_h = 2.5\,s^{-1}$, and extra power roughly halved (Table VI). The first such test, 600 s on USB with no INA219 and the old reusable cookie (Fig. 2, Table VIII), showed the same settling at 2.48–2.49 s^{-1}; it measured counters, not power. COOKIE has no budget and stays flat for as long as the attack runs. Before the bucket empties, gate v2 shows no advantage over COOKIE (paired in one session: ≈+18 against +19.7 mW).

This bound only binds because c_h is a placeholder. A forced verify really costs ≈3–3.5 mJ, not the $c_h = 20$ mJ charged—an overcharge of ≈6–7×. At an honest c_h, the sustainable rate $r/c_h \approx 14$–$17\,s^{-1}$ would exceed this attacker's own $\approx 5\,s^{-1}$, and the bucket would never empty: this follows from (1) and the measured rates. The cut in real drain shown here depends on the placeholder overcharging; the cap on charges holds whatever c_h is.

An insider who echoes is not stopped (Table VII). Earlier insider-echo runs were excluded because the attacker's echoes never fired; with a cached signed HELLO resent unchanged (attacker firmware `389d74f`) they fire ≈2 700 times per run. Under either defence the drain's interval overlaps the undefended one, and the budget never binds: no drops in any gate v2 run, and the bucket never fell below 19.26 of 20 J. The single crypto slot holds admits to ≈2.3 s^{-1}, which charges ≈45 mW against a 50 mW refill, below the 2.5 s^{-1} cap. Each admit costs ≈30 mJ of real power, more than c_h charges. Without an echo the same insider is stopped by either defence. A key-holding insider can read the broadcast cookie, so the echoing case is the one to expect; stopping it needs a different control (per-peer limits or revoking the key), which we do not build or test. These 600 s runs sagged the supply by ≈0.05 V, so at a fixed voltage their drain would read roughly 6–10 mW higher (ESTIMATED); the conclusions do not change.

TABLE VII
INSIDER, FIRMWARE F/G 7AFDEA4, ATTACKER 389D74F, 1–2 NOV (ONE SESSION), $n = 3$

Defence	Attacker	Window	Extra power (mW)	Admits/s
none	insider	600 s	$+73.2 \pm 4.1$	2.38
cookie	insider + echo	600 s	$+58.5 \pm 13.7$	2.03
gate v2	insider + echo	600 s	$+69.0 \pm 9.4$	2.26
cookie	insider, no echo	120 s	$+2.8 \pm 1.0$	0.04
gate v2	insider, no echo	120 s	$+2.4 \pm 1.5$	0.08

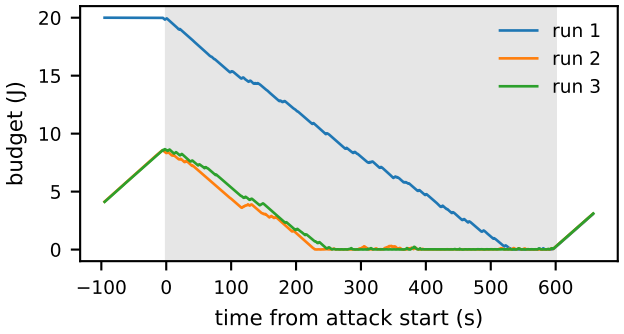


Fig. 2. Field-1's joule budget during three 600 s FORGE+ECHO attacks (shaded) under gate v2, on USB with no INA219 and the earlier reusable cookie (firmware `aca8873`). Once empty, admissions are pinned to the refill rate.

Defences protect energy, not availability. The real gateway's success rate tracks the attacker's radio load, not field-1's crypto load: 97 % with no attack, 49 % at 100 forged frames/s whether field-1 verifies every one (no defence) or none (COOKIE/gate with no echo), and 8–21 % once the attacker echoes. Consistent with frame loss under radio load, though no frame-level loss was measured directly. In a controlled test that moved only the attacker (cookie, same-seq echo, 300 s, $n = 3$ per spot), the gateway succeeded 49 % of the time with the attacker 40 cm away against 17–18 % nearer, while field-1 still admitted ≈4.4 s^{-1}: a more distant attacker drains as much but blocks less. Neither COOKIE nor gate v2 restores availability; the gateway reconnected within a minute after each attack stopped.

IX. ENERGYGATE ON RECORDED TRACES

The shipped tree was trained on SIMULATED traces. We recorded field-1's own eight features and the carried score once a second (6 attacks × 3 sessions plus the 3 budget runs; ≈7 800 rows) and evaluated *leave-one-session-out*—never randomly,

TABLE VIII
BUDGET RUNS: FORGE+ECHO, 600 S, GATE V2, USB, REUSABLE COOKIE (COUNTERS ONLY)

Run	start	empty at	spend/s before	after	drop/s after
1	20.0 J	+537 s	4.52	**2.48**	2.20
2	8.6 J	+235 s	4.13	**2.49**	1.79
3	8.6 J	+260 s	4.10	**2.49**	2.17

TABLE IX
ATTACK DETECTION, LEAVE-ONE-SESSION-OUT (6 FOLDS). DETECTION RATES AT SCORE THRESHOLD 0.5

Model	AUC	TPR@1%	FPR	DRIP	ADAPT.	A.SP.
carried (SIMULATED)	0.876	0.751	0.000	0.00	0.00	0.00
rate rule	0.905	0.809	0.000	0.00	0.82	0.06
tree, 7 feat.	0.935	0.881	0.014	0.05	0.93	0.74
tree, no RSSI	**0.950**	**0.903**	0.008	0.07	0.97	0.86

since neighbouring rows share a 5 s window. Transition rows are excluded; `battery_pct` is constant and dropped. RSSI is 0 when no HELLO arrived, so it can leak "any HELLO = attack"; we report with and without it (Table IX).

The carried SIMULATED-trained model flags FLOOD, INSIDER, FORGE and FORGE+ECHO perfectly with no false alarms, but misses SLOW_DRIP, ADAPTIVE and ADAPTIVE_SPOOF entirely. Retraining on recorded traces lifts AUC to 0.95 and catches most adaptive traffic; dropping RSSI *helps*, consistent with the leak. SLOW_DRIP stays essentially invisible (7 %) to every model. All sessions come from one rig, one room and one distance, so a held-out session is not a held-out environment, and they were recorded before the cookie was bound, with no same-seq echo. The honest reading is C3's: the classifier is evadable. The budget's cap on charges holds regardless, but whether it limits real drain depends on c_h (Section VIII).

Deployed. The no-RSSI tree, refitted on all six sessions, now replaces the SIMULATED model in field-1's firmware (`ml/export_energygate_real.py`). Its exported C scores all 6 968 trace rows within 5×10^{-7} of scikit-learn, with every row on the same side of both policy thresholds. Table X gives what gate v2's thresholds would do with each model's score, left out by session for the retrained tree: almost every ADAPTIVE window moves from spend to challenge, while normal windows are still spent 99 % of the time (a challenged real gateway can still answer the cookie). These are decisions replayed on recorded features, not power measured on the rig with the new firmware, which we have not yet run. ADAPTIVE was also tuned against the old tree; an attacker who knows the new one may find another way under it.

X. LIMITATIONS

- **No battery**: a 5 V adaptor stands in; battery figures are derived from joules, and the victim's radio never sleeps.
- **Echo power does not repeat across sessions.** Extra power under the same-seq echo varied about 2× across sessions with firmware and placement unchanged. A controlled test ruled out signal strength as a simple cause over -24 to -53 dBm (the highest reading, $+19.7$ mW, was at -16 dBm, which that test did not reach). Supply voltage shifts explain some within-session spread but not the between-session gap, whose cause is unknown. Results are therefore compared within a session.
- **Placeholder budget, shown to matter**: every run used $c_h = 20$ mJ, ≈6–7× a forged verify's real cost, so the cut in a forger's drain depends on that overcharge; and ≈0.7× an echoing insider's ≈30 mJ per admit, so the budget never bound against it. Charging 30 mJ with the same r would cap admits at $\approx 1.6\,\mathrm{s}^{-1}$ and empty the bucket in ≈1 000 s under that insider (ESTIMATED, not run). Measured costs are not yet written back, and r is not yet set from a target battery life.
- **No defence against an echoing insider**: neither the cookie nor the gate stops an enrolled key that echoes; this needs per-peer limits or revocation, which we did not build.
- **Attacker energy needs a second monitor we do not have.** We report victim power only; the victim-to-attacker energy ratio is future work, not an ESTIMATED stand-in, since an airtime-based estimate would hide variance a second INA219 would catch.
- **A current trace of one handshake with its phases marked** is future work: the monitor samples at $\approx 989\,\mathrm{s}^{-1}$, enough for a 400 ms handshake, but did not log sample by sample during these runs.
- **Cookie fix**: a first version locked out the legitimate gateway until the gateway's own retry was corrected to keep its (epoch, seq).
- **Firmware differs by table**: Table IV predates the bound cookie; Section IX's traces predate it and contain no same-seq echo.
- **Power results use the SIMULATED-trained model**: the retrained tree is now in the firmware, but every power measurement here predates it, and its effect is shown only by replaying recorded features (Table X).
- **Absolute energy** rests on an unresolved 3.67 % instrument gain difference against a hobby meter; ratios are unaffected.
- **Unauthenticated control channel**, demo keys compiled into images, no formal verification, no level 5, one chip family, one victim and one legitimate peer; legit-success CIs are wide (few handshakes per window).
- **Replay ring**: a captured HELLO whose epoch aged out

TABLE X
SHARE OF ONE-SECOND WINDOWS THE GATE V2 POLICY WOULD SPEND ON ($p \geq 0.7$), BY SCORE. RETRAINED TREE: LEAVE-ONE-SESSION-OUT. REPLAYED FROM RECORDED FEATURES, NOT MEASURED ON THE RIG. "REST" IS FLOOD, INSIDER, FORGE AND FORGE+ECHO

Score	normal	ADAPT.	A.SP.	DRIP	Rest
carried (SIMULATED)	1.00	1.00	0.77	1.00	0.00
retrained, deployed	0.99	0.01	0.14	0.93	0.00

of the 8-entry ring can be replayed once.

XI. Conclusion

On a battery-powered post-quantum responder the battery, not the message, is the attack surface: an enrolled insider raises the victim's power by 15 % and a single forged signature, resent, forces a full ML-DSA verify every time. A first-fragment gate removes the damage cheaply from attackers that neither answer its cookie nor fool its score, but a learned score is evadable, and binding the broadcast cookie to the exact HELLO it answers makes a real echo dearer rather than free. The joule budget always caps what the node charges; it cut real drain only where its charge overstated the true cost, roughly halving a same-seq echo's drain once it ran dry, and it never bound against an enrolled insider who echoes, which nothing here stops. Our defences also protect energy, not availability: the gateway's success tracked the attacker's radio load regardless of which defence ran. Next steps are an honestly-costed budget (with r set from a target battery life), a per-peer control for insiders, measuring the now-deployed retrained gate on the rig, a second monitor for attacker energy, and a duty-cycled victim.

Artifact

`firmware/` (all board firmware, host tests), `ml/` (EnergyGate training/export), `paper/data/` (raw logs, never edited; `paper/runlog.csv` lists every run, excluded ones marked), `paper/analysis/` (scripts rebuilding every table, including `e9_retrain.py` and `fig_budget.py`).

References

[1] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, Aug. 2024.

[2] NIST, "Module-Lattice-Based Digital Signature Standard," FIPS 204, Aug. 2024.

[3] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the cyber attack on the Ukrainian power grid," SANS ICS / E-ISAC, Mar. 2016.

[4] M. J. Kannwischer *et al.*, "Improving software quality in cryptography standardization projects," in *Proc. IEEE EuroS&P Workshops*, 2022. (PQClean.)

[5] Espressif Systems, "ESP-NOW," ESP-IDF Programming Guide.

[6] O. Gillot, W. J. Buchanan, and M. G. Tehrani, "Energy consumption of post-quantum cryptography on constrained and general-purpose architectures," *Cryptography*, vol. 10, no. 4, art. 55, 2026, doi:10.3390/cryptography10040055.

[7] O. Gillot, W. J. Buchanan, and C. Turino, "Testing TLS 1.3 PQC on constrained devices and microcontrollers," in *Proc. QSIoT Workshop (IoT 2026)*, 2026, arXiv:2610.27766.

[8] D. P. Karcz, M. Niemiec, M. A. Kourtis, and T. Köppl, "Exploring hardware implementation feasibility of post-quantum cryptography in embedded systems: Evaluation of NIST-standardized ML-KEM, ML-DSA and SLH-DSA on ESP32-C6," *PLOS ONE*, vol. 21, no. 8, 2026, doi:10.1371/journal.pone.0355179.

[9] F. Segatz and M. I. Al Hafiz, "Efficient implementation of CRYSTALS-KYBER key encapsulation mechanism on ESP32," arXiv:2503.10207, 2025.

[10] F. Ben Ahmed, C. Labrecque, and A. Meddeb, "Benchmarking of post-quantum digital signature schemes on ESP32 and Raspberry Pi platforms," in *Proc. IEEE CCECE*, 2026, pp. 361–366.

[11] L.-F. Lee, Y.-Y. Chang, C.-M. Yu, and K.-H. Yeh, "On the security implications of PQC in TLS: handshake exhaustion and IDS degradation," arXiv:2607.12504, 2026.

[12] N. S. Sammo, S. A. Lura, R. Nowrozy, and S. Bevinakoppa, "Cryptographic DoS amplification in hybrid post-quantum deployments: adversarial algorithm substitution and its countermeasures," *Electronics*, vol. 15, no. 15, art. 3250, 2026.

[13] S. Darzi and A. A. Yavuz, "Privacy-preserving and post-quantum counter denial of service framework for wireless networks," in *Proc. IEEE MILCOM*, 2024, pp. 1076–1081.

[14] R. Hummen, H. Wirtz, J. H. Ziegeldorf, J. Hiller, and K. Wehrle, "Tailoring end-to-end IP security protocols to the Internet of Things," in *Proc. IEEE ICNP*, 2013.

[15] M. Tiloca, C. Gehrmann, and L. Seitz, "On improving resistance to denial of service and key provisioning scalability of the DTLS handshake," *Int. J. Inf. Secur.*, vol. 16, pp. 173–193, 2017.

[16] T. Aura, P. Nikander, and J. Leiwo, "DOS-resistant authentication with client puzzles," in *Security Protocols Workshop*, LNCS, 2001, pp. 170–177.

[17] D. Stebila, L. Kuppusamy, J. Rangasamy, C. Boyd, and J. Gonzalez Nieto, "Stronger difficulty notions for client puzzles and denial-of-service-resistant protocols," in *CT-RSA*, 2011.

[18] Y. Nir and V. Smyslov, "Protecting Internet Key Exchange Protocol Version 2 (IKEv2) implementations from distributed denial-of-service attacks," RFC 8019, 2016.

[19] F. Stajano and R. Anderson, "The resurrecting duckling: security issues for ad-hoc wireless networks," in *Security Protocols Workshop*, 1999.

[20] T. L. Martin, M. S. Hsiao, D. S. Ha, and J. Krishnaswami, "Denial-of-service attacks on battery-powered mobile computers," in *Proc. IEEE PerCom*, 2004, pp. 309–318.

[21] D. Raymond, R. C. Marchany, M. I. Brownfield, and S. F. Midkiff, "Effects of denial-of-sleep attacks on wireless sensor network MAC protocols," *IEEE Trans. Veh. Technol.*, vol. 58, no. 1, pp. 367–380, 2009.

[22] M. Pirretti *et al.*, "The sleep deprivation attack in sensor networks: analysis and methods of defense," *Int. J. Distrib. Sensor Netw.*, vol. 2, no. 3, pp. 267–287, 2006.

[23] G. S. Kuaban, E. Gelenbe, T. Czachórski, P. Czekalski, and J. K. Tangka, "Modelling of the energy depletion process and battery depletion attacks for battery-powered Internet of Things (IoT) devices," *Sensors*, vol. 23, no. 13, art. 6183, 2023.

[24] K.-F. Krentz, C. Meinel, and H. Graupner, "Denial-of-sleep-resilient session key establishment for IEEE 802.15.4 security: from adaptive to responsive," in *Proc. EWSN*, 2018, pp. 25–36.

[25] S. Hristozov, M. Huber, and G. Sigl, "Protecting RESTful IoT devices from battery exhaustion DoS attacks," in *Proc. IEEE HOST*, 2020, pp. 316–327.